

```

In [ ]: # =====
# Project:
# ResNet-50 for Skin Disease Classification
#
# Description:
# An image classification pipeline using a pre-trained ResNet-50 model to classify
# skin diseases. It preloads the entire dataset into RAM for faster training and
# includes early stopping.
#
# Author:
# Gemini (Modified from original ViT-GNN script by MD. Sazib Ahmed)
#
# Date:
# September 24, 2025
# =====

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import os
import numpy as np
import pandas as pd
from tqdm import tqdm
import math
import warnings
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image

# Core ML/DL Libraries
import timm
from torchvision import transforms, datasets
from torch.utils.data import Dataset, DataLoader

# Evaluation & Utilities
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    roc_auc_score,
    cohen_kappa_score,
    matthews_corrcoef,
)
from sklearn.preprocessing import label_binarize

# Suppress non-critical warnings
warnings.filterwarnings("ignore", category=UserWarning, module="PIL")
os.environ["TOKENIZERS_PARALLELISM"] = "false"

# =====
# I. SETUP AND CONFIGURATION

```

```

# =====
config = {
    # --- Hardware & Performance Tuning ---
    "DEVICE": torch.device("cuda" if torch.cuda.is_available() else "cpu"),
    "AMP_ENABLED": True,
    "NUM_WORKERS": 0, # MUST be 0 for in-memory dataset on Windows

    # --- File Paths ---
    "DATA_DIR": r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset",
    "RESULTS_DIR": "full_Dataset_ResNet50",

    # --- Model & Training Hyperparameters ---
    "MODEL_NAME": 'resnet50',
    "IMAGE_SIZE": 224,
    "BATCH_SIZE": 128,
    "NUM_EPOCHS": 200,
    "LEARNING_RATE": 1e-4,
    "WEIGHT_DECAY": 1e-4,
    "WARMUP_EPOCHS": 10,
    "EARLY_STOPPING_PATIENCE": 20,

    # --- Reproducibility ---
    "RANDOM_SEED": 42
}

# --- Create results directory ---
os.makedirs(config["RESULTS_DIR"], exist_ok=True)
config["MODEL_SAVE_PATH_BEST"] = os.path.join(config["RESULTS_DIR"], "ResNet50_best")

# --- Set Seed for Reproducible Experiments ---
def set_seed(seed):
    torch.manual_seed(seed)
    np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = True
    torch.backends.cudnn.deterministic = False

set_seed(config["RANDOM_SEED"])

# --- Initial Setup Verification ---
print("\n" + "="*80)
print(" " * 20 + "ResNet-50 Classification Pipeline Initialization")
print("="*80)
print(f"\n[INFO] PyTorch Version: {torch.__version__}")
print(f"[INFO] Data Loading Mode: In-Memory (Preloading to RAM)")
print(f"[INFO] Using Device: {config['DEVICE']}")
print(f"[INFO] Early Stopping Patience: {config['EARLY_STOPPING_PATIENCE']} epochs")
print(f"[INFO] Training for up to {config['NUM_EPOCHS']} epochs.")
print("="*80)

# =====
# II. DATA PREPARATION & HANDLING

```

```
# =====

# NEW: Custom Dataset for preloaded data
class InMemoryDataset(Dataset):
    def __init__(self, data_list, transform=None):
        """
        data_list: A list of tuples, where each tuple is (image_tensor, label_index)
        """
        self.data_list = data_list
        self.transform = transform

    def __len__(self):
        return len(self.data_list)

    def __getitem__(self, idx):
        image, label = self.data_list[idx]
        if self.transform:
            # Apply GPU-based augmentations if available
            image = self.transform(image)
        return image, label

# Define separate transforms for CPU pre-processing and potential GPU augmentation
# CPU transforms are applied once during pre-loading.
cpu_transforms = transforms.Compose([
    transforms.Resize((config["IMAGE_SIZE"], config["IMAGE_SIZE"])),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# GPU transforms can be applied on-the-fly during training for augmentation.
# This offloads augmentation work from the CPU.
gpu_train_transforms = torch.nn.Sequential(
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    # Note: ColorJitter is often faster on CPU but can be run on GPU
    # transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)
).to(config["DEVICE"])

print("\n" + "="*80)
print(" " * 28 + "Phase 1: Data Preparation")
print("="*80)

# MODIFIED: Preload all data into memory
try:
    # Use ImageFolder just to discover images and labels easily
    temp_datasets = {
        x: datasets.ImageFolder(os.path.join(config["DATA_DIR"], x))
        for x in ['train', 'val', 'test']
    }
    class_names = temp_datasets['train'].classes
    num_classes = len(class_names)

    print(f"[INFO] Discovered {num_classes} classes: {' '.join(class_names)}")
    print("[INFO] Pre-loading all images into RAM. This may take a moment...")

```

```

preloaded_data = {'train': [], 'val': [], 'test': []}
for split in ['train', 'val', 'test']:
    for path, label in tqdm(temp_datasets[split].samples, desc=f"--> Loading {s
        with Image.open(path).convert("RGB") as img:
            image_tensor = cpu_transforms(img)
            preloaded_data[split].append((image_tensor, label))

# Create our custom in-memory datasets
image_datasets = {
    'train': InMemoryDataset(preloaded_data['train'], transform=cpu_train_trans
    'val': InMemoryDataset(preloaded_data['val']), # No augmentation for val/te
    'test': InMemoryDataset(preloaded_data['test'])
}

dataloaders = {
    x: DataLoader(
        image_datasets[x],
        batch_size=config["BATCH_SIZE"],
        shuffle=(x == 'train'),
        num_workers=config["NUM_WORKERS"],
        pin_memory=True # Effective when NUM_WORKERS > 0, but harmless
    )
    for x in ['train', 'val', 'test']
}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val', 'test']}

print(f"[SUCCESS] All image data has been pre-loaded into memory.")
print(f" - Dataset sizes: Train={dataset_sizes['train']}, Val={dataset_sizes['v

except FileNotFoundError as e:
    print(f"\n[ERROR] Dataset not found. Please check the path in config['DATA_DIR']
    print(f"Details: {e}")
    exit()

# =====
# III. MODEL, OPTIMIZER, AND SCHEDULER INITIALIZATION
# =====
model = timm.create_model(config["MODEL_NAME"], pretrained=True, num_classes=num_cl
model.to(config["DEVICE"])

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=config["LEARNING_RATE"], weight_deca

lr_lambda = lambda epoch: (epoch + 1) / config["WARMUP_EPOCHS"] if epoch < config["
    0.5 * (1 + math.cos(math.pi * (epoch - config["WARMUP_EPOCHS"])) / (config["NUM_
scheduler = optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

scaler = torch.amp.GradScaler(device=config["DEVICE"].type, enabled=config["AMP_ENA

# =====
# IV. TRAINING & VALIDATION LOOPS
# =====
def train_one_epoch():
    model.train()
    running_loss = 0.0

```

```

correct_preds = 0
progress_bar = tqdm(dataloaders['train'], desc="--> Training", leave=False)

for inputs, labels in progress_bar:
    inputs = inputs.to(config["DEVICE"], non_blocking=True)
    labels = labels.to(config["DEVICE"], non_blocking=True)

    # Apply GPU augmentations
    inputs = image_datasets['train'].transform(inputs)

    optimizer.zero_grad(set_to_none=True)

    with torch.amp.autocast(device_type=config["DEVICE"].type, dtype=torch.float32):
        outputs = model(inputs)
        loss = criterion(outputs, labels)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()

    _, preds = torch.max(outputs, 1)
    running_loss += loss.item() * inputs.size(0)
    correct_preds += torch.sum(preds == labels.data)
    progress_bar.set_postfix(loss=f"{loss.item():.4f}")

epoch_loss = running_loss / dataset_sizes['train']
epoch_acc = correct_preds.double() / dataset_sizes['train']
return epoch_loss, epoch_acc.item()

def validate_one_epoch():
    model.eval()
    running_loss = 0.0
    correct_preds = 0
    progress_bar = tqdm(dataloaders['val'], desc="--> Validating", leave=False)

    with torch.no_grad():
        for inputs, labels in progress_bar:
            inputs = inputs.to(config["DEVICE"])
            labels = labels.to(config["DEVICE"])

            with torch.amp.autocast(device_type=config["DEVICE"].type, dtype=torch.float32):
                outputs = model(inputs)
                loss = criterion(outputs, labels)

            _, preds = torch.max(outputs, 1)
            running_loss += loss.item() * inputs.size(0)
            correct_preds += torch.sum(preds == labels.data)

    epoch_loss = running_loss / dataset_sizes['val']
    epoch_acc = correct_preds.double() / dataset_sizes['val']
    return epoch_loss, epoch_acc.item()

# =====
# V. EVALUATION & VISUALIZATION FUNCTIONS

```

```

# =====
def plot_training_history(history):
    save_path = os.path.join(config["RESULTS_DIR"], "training_history.png")
    epochs = range(1, len(history['train_loss']) + 1)
    plt.style.use('seaborn-v0_8-darkgrid')
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 7))
    ax1.plot(epochs, history['train_loss'], 'o-', label='Train Loss')
    ax1.plot(epochs, history['val_loss'], 'o-', label='Validation Loss')
    ax1.set_title('Model Loss Over Epochs')
    ax1.set_xlabel('Epochs')
    ax1.set_ylabel('Loss')
    ax1.legend()
    ax2.plot(epochs, [acc * 100 for acc in history['train_acc']], 'o-', label='Train Accuracy')
    ax2.plot(epochs, [acc * 100 for acc in history['val_acc']], 'o-', label='Validation Accuracy')
    ax2.set_title('Model Accuracy Over Epochs')
    ax2.set_xlabel('Epochs')
    ax2.set_ylabel('Accuracy (%)')
    ax2.legend()
    fig.suptitle('ResNet-50 Training & Validation History')
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.savefig(save_path, dpi=300)
    plt.show()
    plt.close()
    print(f"\nSaved training history plot to: {save_path}")

def plot_scalar_metrics(metrics):
    save_path = os.path.join(config["RESULTS_DIR"], "scalar_metrics.png")
    plt.style.use('seaborn-v0_8-whitegrid')
    fig, ax = plt.subplots(figsize=(10, 6))
    metrics_df = pd.DataFrame(list(metrics.items()), columns=['Metric', 'Value'])
    sns.barplot(x='Value', y='Metric', data=metrics_df, ax=ax, palette='viridis', h=0.8)
    ax.set_title('Overall Scalar Performance Metrics')
    ax.set_xlabel('Value')
    ax.set_ylabel('')
    ax.set_xlim(0, 1)
    for p in ax.patches:
        width = p.get_width()
        ax.text(width + 0.01, p.get_y() + p.get_height() / 2, f'{width:.4f}', va='center')
    plt.tight_layout()
    plt.savefig(save_path, dpi=300)
    plt.show()
    plt.close()
    print(f"\nSaved scalar metrics plot to: {save_path}")

def perform_full_evaluation(model_path, loader, class_names):
    print("\n" + "="*80)
    print("OVERALL TEST PERFORMANCE")
    print("="*80)
    try:
        eval_model = timm.create_model(config["MODEL_NAME"], pretrained=False, num_classes=len(class_names))
        eval_model.load_state_dict(torch.load(model_path, map_location=config["DEVICE"]))
        eval_model.to(config["DEVICE"])
        eval_model.eval()
    except FileNotFoundError:

```

```

print(f"\n[ERROR] Model file not found at {model_path}. Evaluation skipped.
return

all_labels, all_preds, all_probs = [], [], []
with torch.no_grad():
    for inputs, labels in tqdm(loader, desc="--> Evaluating on Test Set"):
        inputs = inputs.to(config["DEVICE"])
        outputs = eval_model(inputs)
        probs = F.softmax(outputs, dim=1)
        _, predicted = torch.max(outputs.data, 1)
        all_labels.extend(labels.cpu().numpy())
        all_preds.extend(predicted.cpu().numpy())
        all_probs.extend(probs.cpu().numpy())

all_labels, all_preds, all_probs = np.array(all_labels), np.array(all_preds), np.array(all_probs)
print(classification_report(all_labels, all_preds, target_names=class_names, digits=2))

labels_bin = label_binarize(all_labels, classes=range(len(class_names)))
macro_roc_auc = roc_auc_score(labels_bin, all_probs, average='macro', multi_class='macro')

scalar_metrics = {
    "Accuracy": (all_labels == all_preds).mean(),
    "Macro ROC AUC": macro_roc_auc,
    "Cohen's Kappa": cohen_kappa_score(all_labels, all_preds),
    "Matthews CorrCoef": matthews_corrcoef(all_labels, all_preds),
}

print("\n--- Overall Scalar Metrics ---")
for name, value in scalar_metrics.items(): print(f"{name}: {value:.4f}")

cm_path = os.path.join(config["RESULTS_DIR"], "confusion_matrix.png")
plt.style.use('default')
plt.figure(figsize=(10, 8))
sns.heatmap(confusion_matrix(all_labels, all_preds, normalize='true'), annot=True, cmap='Blues')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Normalized Confusion Matrix')
plt.savefig(cm_path, dpi=300)
plt.show()
plt.close()
print(f"\nSaved normalized confusion matrix to: {cm_path}")

roc_path = os.path.join(config["RESULTS_DIR"], "roc_curves.png")
plt.style.use('seaborn-v0_8-darkgrid')
plt.figure(figsize=(12, 9))
colors = matplotlib.colormaps.get('viridis', len(class_names))
for i, (class_name, color) in enumerate(zip(class_names, colors.colors)):
    fpr, tpr, _ = roc_curve(labels_bin[:, i], all_probs[:, i])
    plt.plot(fpr, tpr, color=color, lw=2.5, label=f'ROC for {class_name} (AUC = {macro_roc_auc:.4f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Chance')
plt.xlim([-0.02, 1.0]); plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curve - One-vs-Rest'); plt.legend(loc="lower right")
plt.savefig(roc_path, dpi=300); plt.show(); plt.close()
print(f"\nSaved ROC curves plot to: {roc_path}")

```

```

plot_scalar_metrics(scalar_metrics)
print("\nDone.")

# =====
# VI. MAIN EXECUTION BLOCK
# =====
if __name__ == '__main__':
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val_loss = float('inf')
    epochs_no_improve = 0
    best_epoch = 0

    print("\n" + "="*80)
    print(" " * 29 + "Phase 2: Model Training")
    print("="*80)
    print(f"[INFO] Starting training for up to {config['NUM_EPOCHS']} epochs...")
    print(f"[INFO] Early stopping is active with patience={config['EARLY_STOPPING_PATIENCE']}")

    for epoch in range(config["NUM_EPOCHS"]):
        train_loss, train_acc = train_one_epoch()
        val_loss, val_acc = validate_one_epoch()
        scheduler.step()

        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)

        current_lr = optimizer.param_groups[0]['lr']

        print(
            f"Epoch [{epoch+1:03d}/{config['NUM_EPOCHS']:03d}] | "
            f"LR: {current_lr:.2e} | "
            f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
            f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}"
        )

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            epochs_no_improve = 0
            best_epoch = epoch + 1
            torch.save(model.state_dict(), config["MODEL_SAVE_PATH_BEST"])
            print(f"    > Validation loss improved to {best_val_loss:.4f}. Saving best model.")
        else:
            epochs_no_improve += 1
            print(f"    > Val loss did not improve for {epochs_no_improve} epoch(s)")

            if epochs_no_improve >= config['EARLY_STOPPING_PATIENCE']:
                print(f"\n[INFO] Early stopping triggered after {epoch + 1} epochs.")
                print(f"    Best model was saved at epoch {best_epoch} with validation loss {best_val_loss:.4f}")
                break

    print(f"\n[SUCCESS] Model training completed.")
    plot_training_history(history)

    perform_full_evaluation(

```



```
model_path=config["MODEL_SAVE_PATH_BEST"],
loader=dataloaders['test'],
class_names=class_names
)
```

ResNet-50 Classification Pipeline Initialization

```
[INFO] PyTorch Version: 2.7.1+cu128
[INFO] Data Loading Mode: In-Memory (Preloading to RAM)
[INFO] Using Device: cuda
[INFO] Early Stopping Patience: 20 epochs
[INFO] Training for up to 200 epochs.
```

Phase 1: Data Preparation

```
[INFO] Discovered 5 classes: Herpes, Melanoma, Monkeypox, Sarampion, Varicela
[INFO] Pre-loading all images into RAM. This may take a moment...
```

```
--> Loading train data: 100%|██████████|  
2000/2000 [00:31<00:00, 62.97it/s]  
--> Loading val data: 100%|██████████|  
| 250/250 [00:01<00:00, 151.62it/s]  
--> Loading test data: 100%|██████████|  
| 500/500 [00:06<00:00, 74.33it/s]
```

[SUCCESS] All image data has been pre-loaded into memory.
- Dataset sizes: Train=2000, Val=250, Test=500

Phase 2: Model Training

```
[INFO] Starting training for up to 200 epochs...
[INFO] Early stopping is active with patience=20.
```

```
Epoch [001/200] | LR: 2.00e-05 | Train Loss: 1.6095 | Train Acc: 0.2085 | Val Loss: 1.6059 | Val Acc: 0.1840
  > Validation loss improved to 1.6059. Saving best model.
```

```
Epoch [002/200] | LR: 3.00e-05 | Train Loss: 1.5991 | Train Acc: 0.2545 | Val Loss: 1.5942 | Val Acc: 0.2440
> Validation loss improved to 1.5942. Saving best model.
```

```
Epoch [003/200] | LR: 4.00e-05 | Train Loss: 1.5817 | Train Acc: 0.3390 | Val Loss: 1.5814 | Val Acc: 0.3320
  > Validation loss improved to 1.5814. Saving best model.
```

```
Epoch [004/200] | LR: 5.00e-05 | Train Loss: 1.5575 | Train Acc: 0.4450 | Val Loss: 1.5577 | Val Acc: 0.4680
> Validation loss improved to 1.5577. Saving best model.
```

```
Epoch [005/200] | LR: 6.00e-05 | Train Loss: 1.5198 | Train Acc: 0.5345 | Val Loss: 1.5249 | Val Acc: 0.5200
  > Validation loss improved to 1.5249. Saving best model.
```

```
Epoch [006/200] | LR: 7.00e-05 | Train Loss: 1.4584 | Train Acc: 0.6125 | Val Loss: 1.4705 | Val Acc: 0.5760
  > Validation loss improved to 1.4705. Saving best model.

Epoch [007/200] | LR: 8.00e-05 | Train Loss: 1.3623 | Train Acc: 0.6880 | Val Loss: 1.3880 | Val Acc: 0.6240
  > Validation loss improved to 1.3880. Saving best model.

Epoch [008/200] | LR: 9.00e-05 | Train Loss: 1.2365 | Train Acc: 0.7115 | Val Loss: 1.2641 | Val Acc: 0.6320
  > Validation loss improved to 1.2641. Saving best model.

Epoch [009/200] | LR: 1.00e-04 | Train Loss: 1.0711 | Train Acc: 0.7395 | Val Loss: 1.1115 | Val Acc: 0.6840
  > Validation loss improved to 1.1115. Saving best model.

Epoch [010/200] | LR: 1.00e-04 | Train Loss: 0.8893 | Train Acc: 0.7635 | Val Loss: 0.9717 | Val Acc: 0.7120
  > Validation loss improved to 0.9717. Saving best model.

Epoch [011/200] | LR: 1.00e-04 | Train Loss: 0.7106 | Train Acc: 0.8135 | Val Loss: 0.8234 | Val Acc: 0.7640
  > Validation loss improved to 0.8234. Saving best model.

Epoch [012/200] | LR: 1.00e-04 | Train Loss: 0.5629 | Train Acc: 0.8590 | Val Loss: 0.6945 | Val Acc: 0.8320
  > Validation loss improved to 0.6945. Saving best model.

Epoch [013/200] | LR: 9.99e-05 | Train Loss: 0.4464 | Train Acc: 0.8965 | Val Loss: 0.5986 | Val Acc: 0.8400
  > Validation loss improved to 0.5986. Saving best model.

Epoch [014/200] | LR: 9.99e-05 | Train Loss: 0.3576 | Train Acc: 0.9095 | Val Loss: 0.5106 | Val Acc: 0.8600
  > Validation loss improved to 0.5106. Saving best model.

Epoch [015/200] | LR: 9.98e-05 | Train Loss: 0.2861 | Train Acc: 0.9190 | Val Loss: 0.4641 | Val Acc: 0.8600
  > Validation loss improved to 0.4641. Saving best model.

Epoch [016/200] | LR: 9.98e-05 | Train Loss: 0.2402 | Train Acc: 0.9350 | Val Loss: 0.4469 | Val Acc: 0.8560
  > Validation loss improved to 0.4469. Saving best model.

Epoch [017/200] | LR: 9.97e-05 | Train Loss: 0.1971 | Train Acc: 0.9455 | Val Loss: 0.4216 | Val Acc: 0.8680
  > Validation loss improved to 0.4216. Saving best model.

Epoch [018/200] | LR: 9.96e-05 | Train Loss: 0.1641 | Train Acc: 0.9580 | Val Loss: 0.4035 | Val Acc: 0.8720
  > Validation loss improved to 0.4035. Saving best model.
```

```
Epoch [019/200] | LR: 9.94e-05 | Train Loss: 0.1358 | Train Acc: 0.9635 | Val Loss: 0.3936 | Val Acc: 0.8640
  > Validation loss improved to 0.3936. Saving best model.

Epoch [020/200] | LR: 9.93e-05 | Train Loss: 0.1209 | Train Acc: 0.9680 | Val Loss: 0.3920 | Val Acc: 0.8760
  > Validation loss improved to 0.3920. Saving best model.

Epoch [021/200] | LR: 9.92e-05 | Train Loss: 0.0968 | Train Acc: 0.9780 | Val Loss: 0.4007 | Val Acc: 0.8720
  > Val loss did not improve for 1 epoch(s).

Epoch [022/200] | LR: 9.90e-05 | Train Loss: 0.0886 | Train Acc: 0.9795 | Val Loss: 0.3955 | Val Acc: 0.8680
  > Val loss did not improve for 2 epoch(s).

Epoch [023/200] | LR: 9.88e-05 | Train Loss: 0.0772 | Train Acc: 0.9795 | Val Loss: 0.3719 | Val Acc: 0.8800
  > Validation loss improved to 0.3719. Saving best model.

Epoch [024/200] | LR: 9.87e-05 | Train Loss: 0.0704 | Train Acc: 0.9815 | Val Loss: 0.3901 | Val Acc: 0.8840
  > Val loss did not improve for 1 epoch(s).

Epoch [025/200] | LR: 9.85e-05 | Train Loss: 0.0619 | Train Acc: 0.9865 | Val Loss: 0.3793 | Val Acc: 0.8800
  > Val loss did not improve for 2 epoch(s).

Epoch [026/200] | LR: 9.83e-05 | Train Loss: 0.0497 | Train Acc: 0.9890 | Val Loss: 0.3953 | Val Acc: 0.8840
  > Val loss did not improve for 3 epoch(s).

Epoch [027/200] | LR: 9.80e-05 | Train Loss: 0.0502 | Train Acc: 0.9860 | Val Loss: 0.3796 | Val Acc: 0.8960
  > Val loss did not improve for 4 epoch(s).

Epoch [028/200] | LR: 9.78e-05 | Train Loss: 0.0437 | Train Acc: 0.9890 | Val Loss: 0.3860 | Val Acc: 0.8960
  > Val loss did not improve for 5 epoch(s).

Epoch [029/200] | LR: 9.76e-05 | Train Loss: 0.0397 | Train Acc: 0.9915 | Val Loss: 0.4064 | Val Acc: 0.8880
  > Val loss did not improve for 6 epoch(s).

Epoch [030/200] | LR: 9.73e-05 | Train Loss: 0.0368 | Train Acc: 0.9910 | Val Loss: 0.4215 | Val Acc: 0.8800
  > Val loss did not improve for 7 epoch(s).

Epoch [031/200] | LR: 9.70e-05 | Train Loss: 0.0366 | Train Acc: 0.9880 | Val Loss: 0.4182 | Val Acc: 0.8760
  > Val loss did not improve for 8 epoch(s).
```

Epoch [032/200] | LR: 9.67e-05 | Train Loss: 0.0336 | Train Acc: 0.9920 | Val Loss: 0.4066 | Val Acc: 0.8920

> Val loss did not improve for 9 epoch(s).

Epoch [033/200] | LR: 9.64e-05 | Train Loss: 0.0283 | Train Acc: 0.9930 | Val Loss: 0.4125 | Val Acc: 0.8840

> Val loss did not improve for 10 epoch(s).

Epoch [034/200] | LR: 9.61e-05 | Train Loss: 0.0248 | Train Acc: 0.9935 | Val Loss: 0.4155 | Val Acc: 0.8880

> Val loss did not improve for 11 epoch(s).

Epoch [035/200] | LR: 9.58e-05 | Train Loss: 0.0202 | Train Acc: 0.9950 | Val Loss: 0.4360 | Val Acc: 0.8920

> Val loss did not improve for 12 epoch(s).

Epoch [036/200] | LR: 9.55e-05 | Train Loss: 0.0241 | Train Acc: 0.9940 | Val Loss: 0.4449 | Val Acc: 0.8920

> Val loss did not improve for 13 epoch(s).

Epoch [037/200] | LR: 9.51e-05 | Train Loss: 0.0223 | Train Acc: 0.9935 | Val Loss: 0.4489 | Val Acc: 0.8960

> Val loss did not improve for 14 epoch(s).

Epoch [038/200] | LR: 9.47e-05 | Train Loss: 0.0218 | Train Acc: 0.9950 | Val Loss: 0.4365 | Val Acc: 0.8880

> Val loss did not improve for 15 epoch(s).

Epoch [039/200] | LR: 9.44e-05 | Train Loss: 0.0176 | Train Acc: 0.9965 | Val Loss: 0.4092 | Val Acc: 0.9000

> Val loss did not improve for 16 epoch(s).

Epoch [040/200] | LR: 9.40e-05 | Train Loss: 0.0216 | Train Acc: 0.9955 | Val Loss: 0.4523 | Val Acc: 0.8960

> Val loss did not improve for 17 epoch(s).

Epoch [041/200] | LR: 9.36e-05 | Train Loss: 0.0182 | Train Acc: 0.9935 | Val Loss: 0.4597 | Val Acc: 0.8800

> Val loss did not improve for 18 epoch(s).

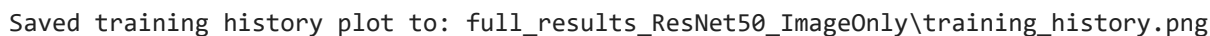
Epoch [042/200] | LR: 9.32e-05 | Train Loss: 0.0194 | Train Acc: 0.9955 | Val Loss: 0.4562 | Val Acc: 0.9000

> Val loss did not improve for 19 epoch(s).

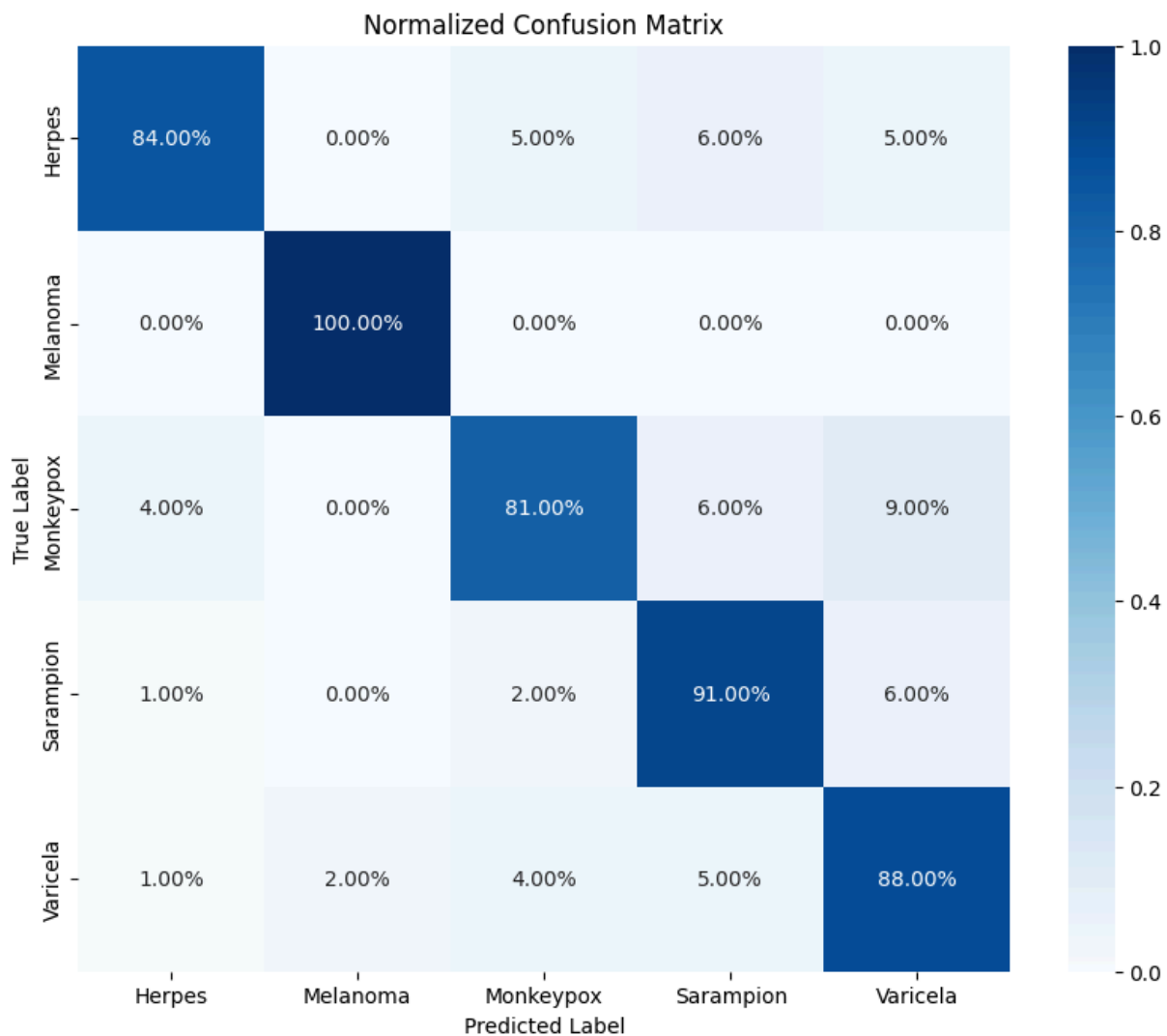
```
> Val loss did not improve for 20 epoch(s).
```

Best model was saved at epoch 23 with validation loss: 0.3719

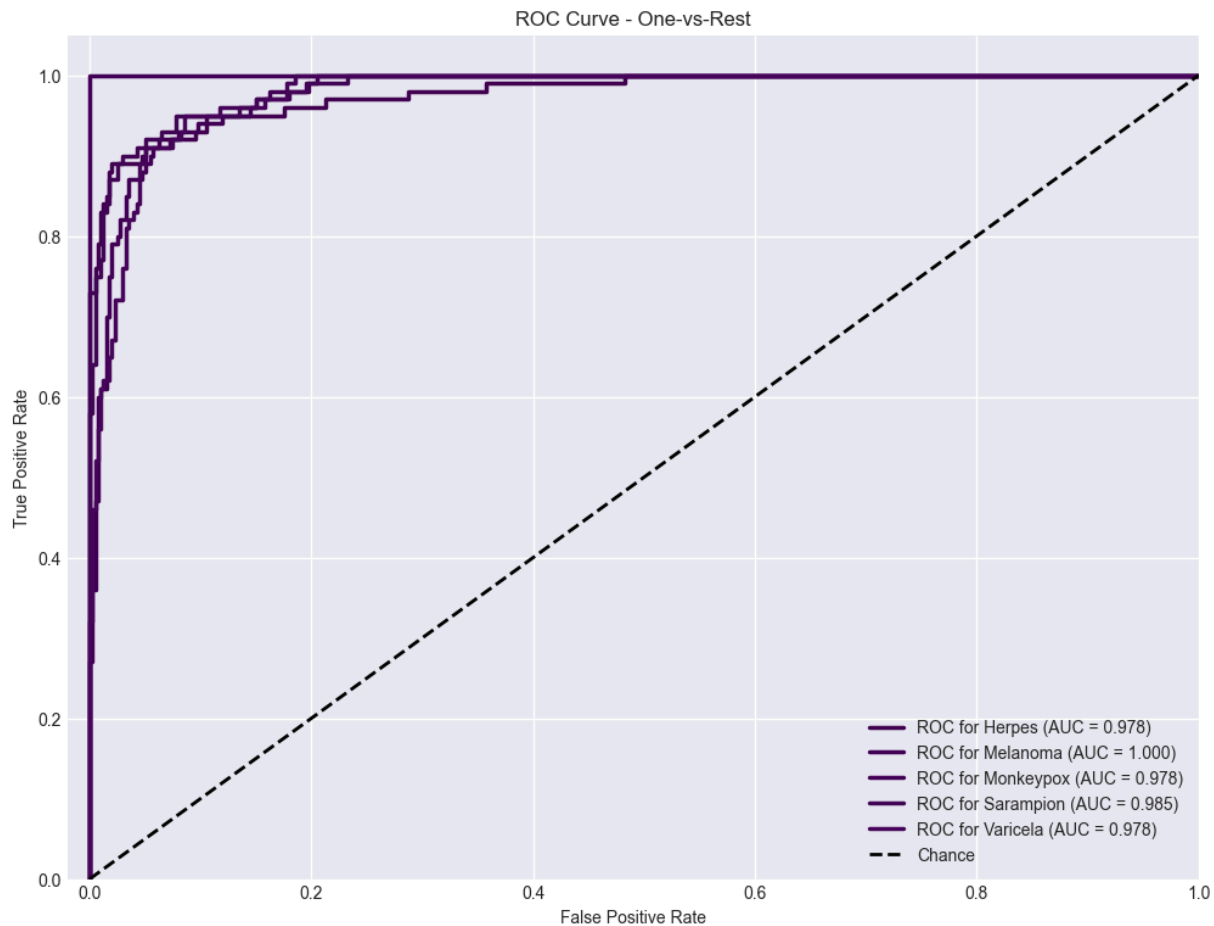
ResNet-50 Training & Validation History



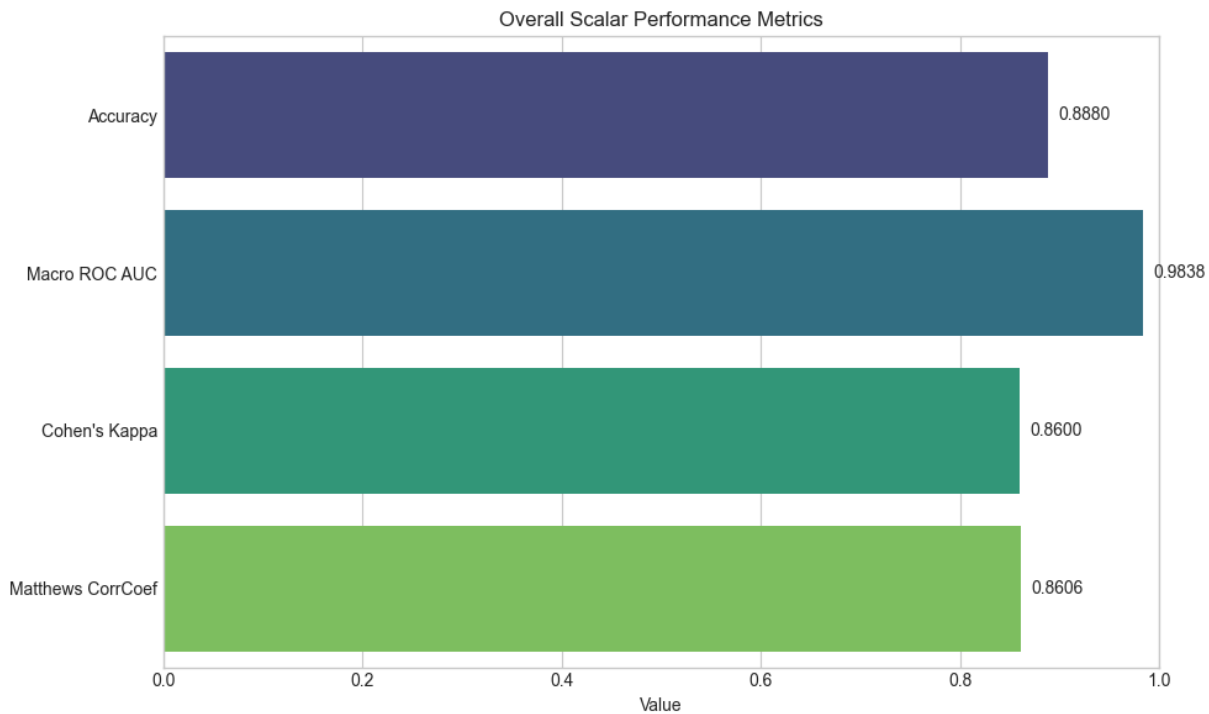
Accuracy: 0.8880
Macro ROC AUC: 0.9838
Cohen's Kappa: 0.8600
Matthews CorrCoef: 0.8606



Saved normalized confusion matrix to: full_results_ResNet50_ImageOnly\confusion_matrix.png



Saved ROC curves plot to: full_results_ResNet50_ImageOnly\roc_curves.png



Saved scalar metrics plot to: full_results_ResNet50_ImageOnly\scalar_metrics.png

Done.

In [1]:

```
# =====  
# Project:
```

```

# ResNet-50 for Skin Disease Classification
#
# Description:
# An image classification pipeline using a pre-trained ResNet-50 model to classify
# skin diseases from a pre-split image dataset. Includes early stopping, dropout,
# and pre-loads the entire dataset into RAM for faster I/O.
#
# Author:
# Gemini (Modified from original ViT-GNN script by MD. Sazib Ahmed)
#
# Date:
# September 24, 2025
# =====

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import os
import numpy as np
import pandas as pd
from tqdm import tqdm
import math
import warnings
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns

# Core ML/DL Libraries
import timm
from torchvision import transforms, datasets

# Evaluation & Utilities
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    roc_auc_score,
    cohen_kappa_score,
    matthews_corrcoef,
)
from sklearn.preprocessing import label_binarize

# Suppress non-critical warnings
warnings.filterwarnings("ignore", category=UserWarning, module="PIL")
os.environ["TOKENIZERS_PARALLELISM"] = "false"

# =====
# I. SETUP AND CONFIGURATION
# =====

config = {
    # --- Hardware & Performance Tuning ---
    "DEVICE": torch.device("cuda" if torch.cuda.is_available() else "cpu"),
    "AMP_ENABLED": True,
    "NUM_WORKERS": 0, # Must be 0 for in-memory datasets

```



```

# --- File Paths ---
"DATA_DIR": r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset",
"RESULTS_DIR": "full_results_ResNet50_ImageOnly_dropout",

# --- Model & Training Hyperparameters ---
"MODEL_NAME": 'resnet50',
"IMAGE_SIZE": 224,
"BATCH_SIZE": 128,
"NUM_EPOCHS": 200,
"LEARNING_RATE": 1e-4,
"WEIGHT_DECAY": 1e-4,
"WARMUP_EPOCHS": 10,
"EARLY_STOPPING_PATIENCE": 20,
"DROPOUT_RATE": 0.5,

# --- Reproducibility ---
"RANDOM_SEED": 42
}

# --- Create results directory ---
os.makedirs(config["RESULTS_DIR"], exist_ok=True)
config["MODEL_SAVE_PATH_BEST"] = os.path.join(config["RESULTS_DIR"], "ResNet50_best")

# --- Set Seed for Reproducible Experiments ---
def set_seed(seed):
    torch.manual_seed(seed)
    np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = True
    torch.backends.cudnn.deterministic = False

set_seed(config["RANDOM_SEED"])

# --- Initial Setup Verification ---
print("\n" + "="*80)
print(" " * 20 + "ResNet-50 Classification Pipeline Initialization")
print("="*80)
print(f"\n[INFO] PyTorch Version: {torch.__version__}")
print(f"[INFO] Data Loading Mode: In-Memory (Pre-loading to RAM)")
print(f"[INFO] Using Device: {config['DEVICE']}")
print(f"[INFO] Dropout Rate: {config['DROPOUT_RATE']}")
print(f"[INFO] Early Stopping Patience: {config['EARLY_STOPPING_PATIENCE']} epochs")
print(f"[INFO] Training for up to {config['NUM_EPOCHS']} epochs.")
print("="*80)

# =====
# II. DATA PREPARATION & HANDLING
# =====

# Data transforms for CPU pre-processing and GPU augmentation.
data_transforms = {

```

```

'cpu_initial': transforms.Compose([
    transforms.Resize((config["IMAGE_SIZE"], config["IMAGE_SIZE"])),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
]),
'gpu_augmentation': nn.Sequential(
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
).to(config["DEVICE"])
}

# Custom Dataset for in-memory data.
class InMemoryDataset(torch.utils.data.Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]

# CORRECTED: Function to Load dataset into memory.
def load_data_in_memory(data_dir, transform):
    print("[INFO] Pre-loading all images into RAM. This may take a moment...")
    data = {}
    labels = {}

    # Use the 'train' split to discover class names
    train_dataset_for_classes = datasets.ImageFolder(os.path.join(data_dir, 'train'))
    class_names = train_dataset_for_classes.classes
    print(f"[INFO] Discovered {len(class_names)} classes: {' '.join(class_names)}")

    for split in ['train', 'val', 'test']:
        # Create an ImageFolder dataset WITHOUT the final tensor transform
        pil_dataset = datasets.ImageFolder(os.path.join(data_dir, split))

        # Manually iterate and transform each image
        data[split] = torch.stack([transform(img) for img, _ in tqdm(pil_dataset, data_dir=split)])
        labels[split] = torch.tensor([label for _, label in pil_dataset])

        print(f"> {split} set loaded: {len(labels[split])} images")

    print("[SUCCESS] All image data has been pre-loaded into memory.")
    return data, labels, class_names

print("\n" + "="*80)
print(" " * 28 + "Phase 1: Data Preparation")
print("="*80)

try:
    # Use the corrected in-memory loading function
    data, labels, class_names = load_data_in_memory(config["DATA_DIR"], data_transform)

```

```

image_datasets = {
    split: InMemoryDataset(data[split], labels[split])
    for split in ['train', 'val', 'test']
}

dataloaders = {
    x: torch.utils.data.DataLoader(
        image_datasets[x],
        batch_size=config["BATCH_SIZE"],
        shuffle=(x == 'train'),
        num_workers=0,
        pin_memory=False # Not needed as data is already tensors
    )
    for x in ['train', 'val', 'test']
}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val', 'test']}
num_classes = len(class_names)

print(f" - Dataset sizes: Train={dataset_sizes['train']}, Val={dataset_sizes['v

except FileNotFoundError as e:
    print(f"\n[ERROR] Dataset not found. Please check the path in config['DATA_DIR']")
    print(f"Details: {e}")
    exit()

# =====
# III. MODEL, OPTIMIZER, AND SCHEDULER INITIALIZATION
# =====
model = timm.create_model(
    config["MODEL_NAME"],
    pretrained=True,
    num_classes=num_classes,
    drop_rate=config["DROPOUT_RATE"]
)
model.to(config["DEVICE"])

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=config["LEARNING_RATE"], weight_deca

lr_lambda = lambda epoch: (epoch + 1) / config["WARMUP_EPOCHS"] if epoch < config["
    0.5 * (1 + math.cos(math.pi * (epoch - config["WARMUP_EPOCHS"])) / (config["NUM_
scheduler = optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

scaler = torch.amp.GradScaler(device=config["DEVICE"].type, enabled=config["AMP_ENA

# =====
# IV. TRAINING & VALIDATION LOOPS
# =====
def train_one_epoch():
    model.train()
    running_loss = 0.0
    correct_preds = 0
    progress_bar = tqdm(dataloaders['train'], desc="--> Training", leave=False)

    for inputs, labels in progress_bar:

```

```

inputs = inputs.to(config["DEVICE"])
labels = labels.to(config["DEVICE"])

# Apply GPU-side augmentations
inputs = data_transforms['gpu_augmentation'](inputs)

optimizer.zero_grad()

with torch.amp.autocast(device_type=config["DEVICE"].type, dtype=torch.float32):
    outputs = model(inputs)
    loss = criterion(outputs, labels)

scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()

_, preds = torch.max(outputs, 1)
running_loss += loss.item() * inputs.size(0)
correct_preds += torch.sum(preds == labels.data)
progress_bar.set_postfix(loss=f"{loss.item():.4f}")

epoch_loss = running_loss / dataset_sizes['train']
epoch_acc = correct_preds.double() / dataset_sizes['train']
return epoch_loss, epoch_acc.item()

def validate_one_epoch():
    model.eval()
    running_loss = 0.0
    correct_preds = 0
    progress_bar = tqdm(dataloaders['val'], desc="--> Validating", leave=False)

    with torch.no_grad():
        for inputs, labels in progress_bar:
            inputs = inputs.to(config["DEVICE"])
            labels = labels.to(config["DEVICE"])

            with torch.amp.autocast(device_type=config["DEVICE"].type, dtype=torch.float32):
                outputs = model(inputs)
                loss = criterion(outputs, labels)

            _, preds = torch.max(outputs, 1)
            running_loss += loss.item() * inputs.size(0)
            correct_preds += torch.sum(preds == labels.data)

    epoch_loss = running_loss / dataset_sizes['val']
    epoch_acc = correct_preds.double() / dataset_sizes['val']
    return epoch_loss, epoch_acc.item()

# =====
# V. EVALUATION & VISUALIZATION FUNCTIONS (No changes needed in these functions)
# =====
def plot_training_history(history):
    save_path = os.path.join(config["RESULTS_DIR"], "training_history.png")
    epochs = range(1, len(history['train_loss']) + 1)

```

```

plt.style.use('seaborn-v0_8-darkgrid')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 7))
ax1.plot(epochs, history['train_loss'], 'o-', label='Train Loss')
ax1.plot(epochs, history['val_loss'], 'o-', label='Validation Loss')
ax1.set_title('Model Loss Over Epochs')
ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
ax1.legend()
ax2.plot(epochs, [acc * 100 for acc in history['train_acc']], 'o-', label='Train Accuracy')
ax2.plot(epochs, [acc * 100 for acc in history['val_acc']], 'o-', label='Validation Accuracy')
ax2.set_title('Model Accuracy Over Epochs')
ax2.set_xlabel('Epochs')
ax2.set_ylabel('Accuracy (%)')
ax2.legend()
fig.suptitle('ResNet-50 Training & Validation History')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(save_path, dpi=300)
plt.show()
plt.close()
print(f"\nSaved training history plot to: {save_path}")

def plot_scalar_metrics(metrics):
    save_path = os.path.join(config["RESULTS_DIR"], "scalar_metrics.png")
    plt.style.use('seaborn-v0_8-whitegrid')
    fig, ax = plt.subplots(figsize=(10, 6))
    metrics_df = pd.DataFrame(list(metrics.items()), columns=['Metric', 'Value'])
    sns.barplot(x='Value', y='Metric', data=metrics_df, ax=ax, palette='viridis', h=0.8)
    ax.set_title('Overall Scalar Performance Metrics')
    ax.set_xlabel('Value')
    ax.set_ylabel('')
    ax.set_xlim(0, 1)
    for p in ax.patches:
        width = p.get_width()
        ax.text(width + 0.01, p.get_y() + p.get_height() / 2, f'{width:.4f}', va='center')
    plt.tight_layout()
    plt.savefig(save_path, dpi=300)
    plt.show()
    plt.close()
    print(f"\nSaved scalar metrics plot to: {save_path}")

def perform_full_evaluation(model_path, loader, class_names):
    print("\n" + "="*80)
    print("OVERALL TEST PERFORMANCE")
    print("="*80)
    try:
        eval_model = timm.create_model(config["MODEL_NAME"], pretrained=False, num_classes=len(class_names))
        eval_model.load_state_dict(torch.load(model_path, map_location=config["DEVICE"]))
        eval_model.to(config["DEVICE"])
        eval_model.eval()
    except FileNotFoundError:
        print(f"\n[ERROR] Model file not found at {model_path}. Evaluation skipped.")
        return

    all_labels, all_preds, all_probs = [], [], []

```

```

with torch.no_grad():
    for inputs, labels in tqdm(loader, desc="--> Evaluating on Test Set"):
        inputs = inputs.to(config["DEVICE"])
        outputs = eval_model(inputs)
        probs = F.softmax(outputs, dim=1)
        _, predicted = torch.max(outputs.data, 1)
        all_labels.extend(labels.cpu().numpy())
        all_preds.extend(predicted.cpu().numpy())
        all_probs.extend(probs.cpu().numpy())

all_labels, all_preds, all_probs = np.array(all_labels), np.array(all_preds), np.array(all_probs)
print(classification_report(all_labels, all_preds, target_names=class_names, digits=2))

labels_bin = label_binarize(all_labels, classes=range(len(class_names)))
macro_roc_auc = roc_auc_score(labels_bin, all_probs, average='macro', multi_class='ovo')

scalar_metrics = {
    "Accuracy": (all_labels == all_preds).mean(),
    "Macro ROC AUC": macro_roc_auc,
    "Cohen's Kappa": cohen_kappa_score(all_labels, all_preds),
    "Matthews CorrCoef": matthews_corrcoef(all_labels, all_preds),
}

print("\n--- Overall Scalar Metrics ---")
for name, value in scalar_metrics.items(): print(f"{name}: {value:.4f}")

cm_path = os.path.join(config["RESULTS_DIR"], "confusion_matrix.png")
plt.style.use('default')
plt.figure(figsize=(10, 8))
sns.heatmap(confusion_matrix(all_labels, all_preds, normalize='true'), annot=True, cmap='Blues')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Normalized Confusion Matrix')
plt.savefig(cm_path, dpi=300)
plt.show()
plt.close()
print(f"\nSaved normalized confusion matrix to: {cm_path}")

roc_path = os.path.join(config["RESULTS_DIR"], "roc_curves.png")
plt.style.use('seaborn-v0_8-darkgrid')
plt.figure(figsize=(12, 9))
colors = matplotlib.colormaps.get('viridis', len(class_names))
for i, (class_name, color) in enumerate(zip(class_names, colors.colors)):
    fpr, tpr, _ = roc_curve(labels_bin[:, i], all_probs[:, i])
    plt.plot(fpr, tpr, color=color, lw=2.5, label=f'ROC for {class_name} (AUC = {roc_auc_score(labels_bin[:, i], all_probs[:, i]):.4f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Chance')
plt.xlim([-0.02, 1.0]); plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curve - One-vs-Rest'); plt.legend(loc="lower right")
plt.savefig(roc_path, dpi=300); plt.show(); plt.close()
print(f"\nSaved ROC curves plot to: {roc_path}")

plot_scalar_metrics(scalar_metrics)
print("\nDone.")

```

```
# =====
```

```

# VI. MAIN EXECUTION BLOCK
# =====
if __name__ == '__main__':
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val_loss = float('inf')
    epochs_no_improve = 0
    best_epoch = 0

    print("\n" + "="*80)
    print(" " * 29 + "Phase 2: Model Training")
    print("="*80)
    print(f"[INFO] Starting training for up to {config['NUM_EPOCHS']} epochs...")
    print(f"[INFO] Early stopping is active with patience={config['EARLY_STOPPING_PATIENCE']}")

    for epoch in range(config["NUM_EPOCHS"]):
        train_loss, train_acc = train_one_epoch()
        val_loss, val_acc = validate_one_epoch()
        scheduler.step()

        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)

        current_lr = optimizer.param_groups[0]['lr']

        print(
            f"Epoch [{epoch+1:03d}/{config['NUM_EPOCHS']:03d}] | "
            f"LR: {current_lr:.2e} | "
            f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
            f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}"
        )

        # Early stopping logic
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            epochs_no_improve = 0
            best_epoch = epoch + 1
            torch.save(model.state_dict(), config["MODEL_SAVE_PATH_BEST"])
            print(f"    > Validation loss improved to {best_val_loss:.4f}. Saving best model")
        else:
            epochs_no_improve += 1
            print(f"    > Val loss did not improve for {epochs_no_improve} epoch(s)")

        if epochs_no_improve >= config['EARLY_STOPPING_PATIENCE']:
            print(f"\n[INFO] Early stopping triggered after {epoch + 1} epochs.")
            print(f"    Best model was saved at epoch {best_epoch} with validation loss {best_val_loss:.4f}")
            break

    print(f"\n[SUCCESS] Model training completed.")
    plot_training_history(history)

    # --- Final Evaluation on Test Set ---
    perform_full_evaluation(
        model_path=config["MODEL_SAVE_PATH_BEST"],
        loader=dataloaders['test'],

```


Epoch [005/200] | LR: 6.00e-05 | Train Loss: 1.5443 | Train Acc: 0.4430 | Val Loss: 1.5402 | Val Acc: 0.5200

> Validation loss improved to 1.5402. Saving best model.

Epoch [006/200] | LR: 7.00e-05 | Train Loss: 1.5006 | Train Acc: 0.5290 | Val Loss: 1.5049 | Val Acc: 0.5680

> Validation loss improved to 1.5049. Saving best model.

Epoch [007/200] | LR: 8.00e-05 | Train Loss: 1.4434 | Train Acc: 0.5755 | Val Loss: 1.4432 | Val Acc: 0.5960

> Validation loss improved to 1.4432. Saving best model.

Epoch [008/200] | LR: 9.00e-05 | Train Loss: 1.3577 | Train Acc: 0.6225 | Val Loss: 1.3520 | Val Acc: 0.6280

> Validation loss improved to 1.3520. Saving best model.

Epoch [009/200] | LR: 1.00e-04 | Train Loss: 1.2370 | Train Acc: 0.6925 | Val Loss: 1.2427 | Val Acc: 0.6440

> Validation loss improved to 1.2427. Saving best model.

Epoch [010/200] | LR: 1.00e-04 | Train Loss: 1.0929 | Train Acc: 0.7285 | Val Loss: 1.1210 | Val Acc: 0.6640

> Validation loss improved to 1.1210. Saving best model.

Epoch [011/200] | LR: 1.00e-04 | Train Loss: 0.9443 | Train Acc: 0.7465 | Val Loss: 1.0081 | Val Acc: 0.6920

> Validation loss improved to 1.0081. Saving best model.

Epoch [012/200] | LR: 1.00e-04 | Train Loss: 0.7933 | Train Acc: 0.7785 | Val Loss: 0.8968 | Val Acc: 0.7560

> Validation loss improved to 0.8968. Saving best model.

Epoch [013/200] | LR: 9.99e-05 | Train Loss: 0.6617 | Train Acc: 0.8220 | Val Loss: 0.7679 | Val Acc: 0.7880

> Validation loss improved to 0.7679. Saving best model.

Epoch [014/200] | LR: 9.99e-05 | Train Loss: 0.5343 | Train Acc: 0.8680 | Val Loss: 0.6761 | Val Acc: 0.8480

> Validation loss improved to 0.6761. Saving best model.

Epoch [015/200] | LR: 9.98e-05 | Train Loss: 0.4490 | Train Acc: 0.8910 | Val Loss: 0.6047 | Val Acc: 0.8400

> Validation loss improved to 0.6047. Saving best model.

Epoch [016/200] | LR: 9.98e-05 | Train Loss: 0.3583 | Train Acc: 0.9160 | Val Loss: 0.5366 | Val Acc: 0.8480

> Validation loss improved to 0.5366. Saving best model.

Epoch [017/200] | LR: 9.97e-05 | Train Loss: 0.3008 | Train Acc: 0.9295 | Val Loss: 0.4734 | Val Acc: 0.8680

> Validation loss improved to 0.4734. Saving best model.

Epoch [018/200] | LR: 9.96e-05 | Train Loss: 0.2474 | Train Acc: 0.9400 | Val Loss: 0.4383 | Val Acc: 0.8600

> Validation loss improved to 0.4383. Saving best model.

Epoch [019/200] | LR: 9.94e-05 | Train Loss: 0.2016 | Train Acc: 0.9495 | Val Loss: 0.4189 | Val Acc: 0.8720

> Validation loss improved to 0.4189. Saving best model.

Epoch [020/200] | LR: 9.93e-05 | Train Loss: 0.1727 | Train Acc: 0.9605 | Val Loss: 0.3988 | Val Acc: 0.8800

> Validation loss improved to 0.3988. Saving best model.

Epoch [021/200] | LR: 9.92e-05 | Train Loss: 0.1412 | Train Acc: 0.9675 | Val Loss: 0.3899 | Val Acc: 0.8680

> Validation loss improved to 0.3899. Saving best model.

Epoch [022/200] | LR: 9.90e-05 | Train Loss: 0.1225 | Train Acc: 0.9660 | Val Loss: 0.3745 | Val Acc: 0.8760

> Validation loss improved to 0.3745. Saving best model.

Epoch [023/200] | LR: 9.88e-05 | Train Loss: 0.1083 | Train Acc: 0.9735 | Val Loss: 0.3796 | Val Acc: 0.8840

> Val loss did not improve for 1 epoch(s).

Epoch [024/200] | LR: 9.87e-05 | Train Loss: 0.0915 | Train Acc: 0.9790 | Val Loss: 0.3828 | Val Acc: 0.8880

> Val loss did not improve for 2 epoch(s).

Epoch [025/200] | LR: 9.85e-05 | Train Loss: 0.0807 | Train Acc: 0.9815 | Val Loss: 0.3830 | Val Acc: 0.8720

> Val loss did not improve for 3 epoch(s).

Epoch [026/200] | LR: 9.83e-05 | Train Loss: 0.0735 | Train Acc: 0.9830 | Val Loss: 0.3680 | Val Acc: 0.8920

> Validation loss improved to 0.3680. Saving best model.

Epoch [027/200] | LR: 9.80e-05 | Train Loss: 0.0616 | Train Acc: 0.9860 | Val Loss: 0.3792 | Val Acc: 0.8960

> Val loss did not improve for 1 epoch(s).

Epoch [028/200] | LR: 9.78e-05 | Train Loss: 0.0573 | Train Acc: 0.9855 | Val Loss: 0.3558 | Val Acc: 0.8960

> Validation loss improved to 0.3558. Saving best model.

Epoch [029/200] | LR: 9.76e-05 | Train Loss: 0.0513 | Train Acc: 0.9875 | Val Loss: 0.3579 | Val Acc: 0.9080

> Val loss did not improve for 1 epoch(s).

Epoch [030/200] | LR: 9.73e-05 | Train Loss: 0.0529 | Train Acc: 0.9840 | Val Loss: 0.3581 | Val Acc: 0.9040

> Val loss did not improve for 2 epoch(s).

Epoch [031/200] | LR: 9.70e-05 | Train Loss: 0.0474 | Train Acc: 0.9870 | Val Loss: 0.3743 | Val Acc: 0.8920

> Val loss did not improve for 3 epoch(s).

Epoch [032/200] | LR: 9.67e-05 | Train Loss: 0.0392 | Train Acc: 0.9885 | Val Loss: 0.3493 | Val Acc: 0.9000

> Validation loss improved to 0.3493. Saving best model.

Epoch [033/200] | LR: 9.64e-05 | Train Loss: 0.0354 | Train Acc: 0.9940 | Val Loss: 0.3659 | Val Acc: 0.9040

> Val loss did not improve for 1 epoch(s).

Epoch [034/200] | LR: 9.61e-05 | Train Loss: 0.0311 | Train Acc: 0.9925 | Val Loss: 0.3925 | Val Acc: 0.8960

> Val loss did not improve for 2 epoch(s).

Epoch [035/200] | LR: 9.58e-05 | Train Loss: 0.0245 | Train Acc: 0.9945 | Val Loss: 0.3857 | Val Acc: 0.9120

> Val loss did not improve for 3 epoch(s).

Epoch [036/200] | LR: 9.55e-05 | Train Loss: 0.0258 | Train Acc: 0.9940 | Val Loss: 0.3857 | Val Acc: 0.8920

> Val loss did not improve for 4 epoch(s).

Epoch [037/200] | LR: 9.51e-05 | Train Loss: 0.0269 | Train Acc: 0.9940 | Val Loss: 0.3853 | Val Acc: 0.8920

> Val loss did not improve for 5 epoch(s).

Epoch [038/200] | LR: 9.47e-05 | Train Loss: 0.0246 | Train Acc: 0.9950 | Val Loss: 0.3968 | Val Acc: 0.8960

> Val loss did not improve for 6 epoch(s).

Epoch [039/200] | LR: 9.44e-05 | Train Loss: 0.0205 | Train Acc: 0.9960 | Val Loss: 0.3982 | Val Acc: 0.8920

> Val loss did not improve for 7 epoch(s).

Epoch [040/200] | LR: 9.40e-05 | Train Loss: 0.0219 | Train Acc: 0.9945 | Val Loss: 0.3713 | Val Acc: 0.9000

> Val loss did not improve for 8 epoch(s).

Epoch [041/200] | LR: 9.36e-05 | Train Loss: 0.0164 | Train Acc: 0.9970 | Val Loss: 0.4038 | Val Acc: 0.9040

> Val loss did not improve for 9 epoch(s).

Epoch [042/200] | LR: 9.32e-05 | Train Loss: 0.0156 | Train Acc: 0.9980 | Val Loss: 0.4174 | Val Acc: 0.8960

> Val loss did not improve for 10 epoch(s).

Epoch [043/200] | LR: 9.27e-05 | Train Loss: 0.0190 | Train Acc: 0.9965 | Val Loss: 0.4393 | Val Acc: 0.8920

> Val loss did not improve for 11 epoch(s).

Epoch [044/200] | LR: 9.23e-05 | Train Loss: 0.0131 | Train Acc: 0.9970 | Val Loss: 0.4253 | Val Acc: 0.8880

> Val loss did not improve for 12 epoch(s).

Epoch [045/200] | LR: 9.19e-05 | Train Loss: 0.0136 | Train Acc: 0.9980 | Val Loss: 0.4451 | Val Acc: 0.8880

> Val loss did not improve for 13 epoch(s).

Epoch [046/200] | LR: 9.14e-05 | Train Loss: 0.0155 | Train Acc: 0.9975 | Val Loss: 0.4110 | Val Acc: 0.8960

> Val loss did not improve for 14 epoch(s).

Epoch [047/200] | LR: 9.09e-05 | Train Loss: 0.0101 | Train Acc: 0.9980 | Val Loss: 0.4059 | Val Acc: 0.8960

> Val loss did not improve for 15 epoch(s).

Epoch [048/200] | LR: 9.05e-05 | Train Loss: 0.0135 | Train Acc: 0.9980 | Val Loss: 0.4123 | Val Acc: 0.8960

> Val loss did not improve for 16 epoch(s).

Epoch [049/200] | LR: 9.00e-05 | Train Loss: 0.0092 | Train Acc: 0.9985 | Val Loss: 0.4101 | Val Acc: 0.9040

> Val loss did not improve for 17 epoch(s).

Epoch [050/200] | LR: 8.95e-05 | Train Loss: 0.0102 | Train Acc: 0.9970 | Val Loss: 0.4201 | Val Acc: 0.9080

> Val loss did not improve for 18 epoch(s).

Epoch [051/200] | LR: 8.89e-05 | Train Loss: 0.0079 | Train Acc: 0.9980 | Val Loss: 0.4327 | Val Acc: 0.8960

> Val loss did not improve for 19 epoch(s).

Epoch [052/200] | LR: 8.84e-05 | Train Loss: 0.0108 | Train Acc: 0.9980 | Val Loss: 0.4276 | Val Acc: 0.8960

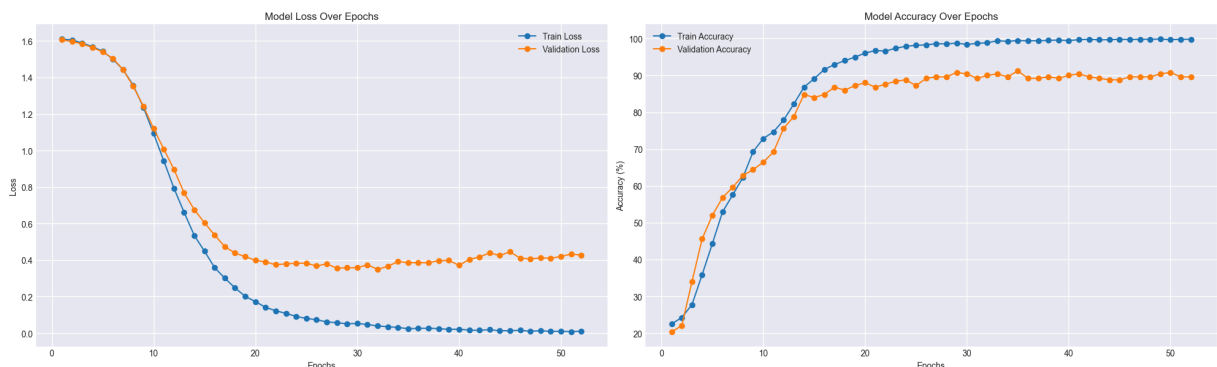
> Val loss did not improve for 20 epoch(s).

[INFO] Early stopping triggered after 52 epochs.

Best model was saved at epoch 32 with validation loss: 0.3493

[SUCCESS] Model training completed.

ResNet-50 Training & Validation History



Saved training history plot to: full_results_ResNet50_ImageOnly_dropout\training_history.png

OVERALL TEST PERFORMANCE

```
--> Evaluating on Test Set: 100%|██████████| 4/4 [00:04<00:00, 1.00s/it]
```

	precision	recall	f1-score	support
Herpes	0.96	0.86	0.91	100
Melanoma	0.98	1.00	0.99	100
Monkeypox	0.86	0.79	0.82	100
Sarampion	0.81	0.91	0.86	100
Varicela	0.82	0.85	0.83	100
accuracy			0.88	500
macro avg	0.88	0.88	0.88	500
weighted avg	0.88	0.88	0.88	500

```

--- Overall Scalar Metrics ---

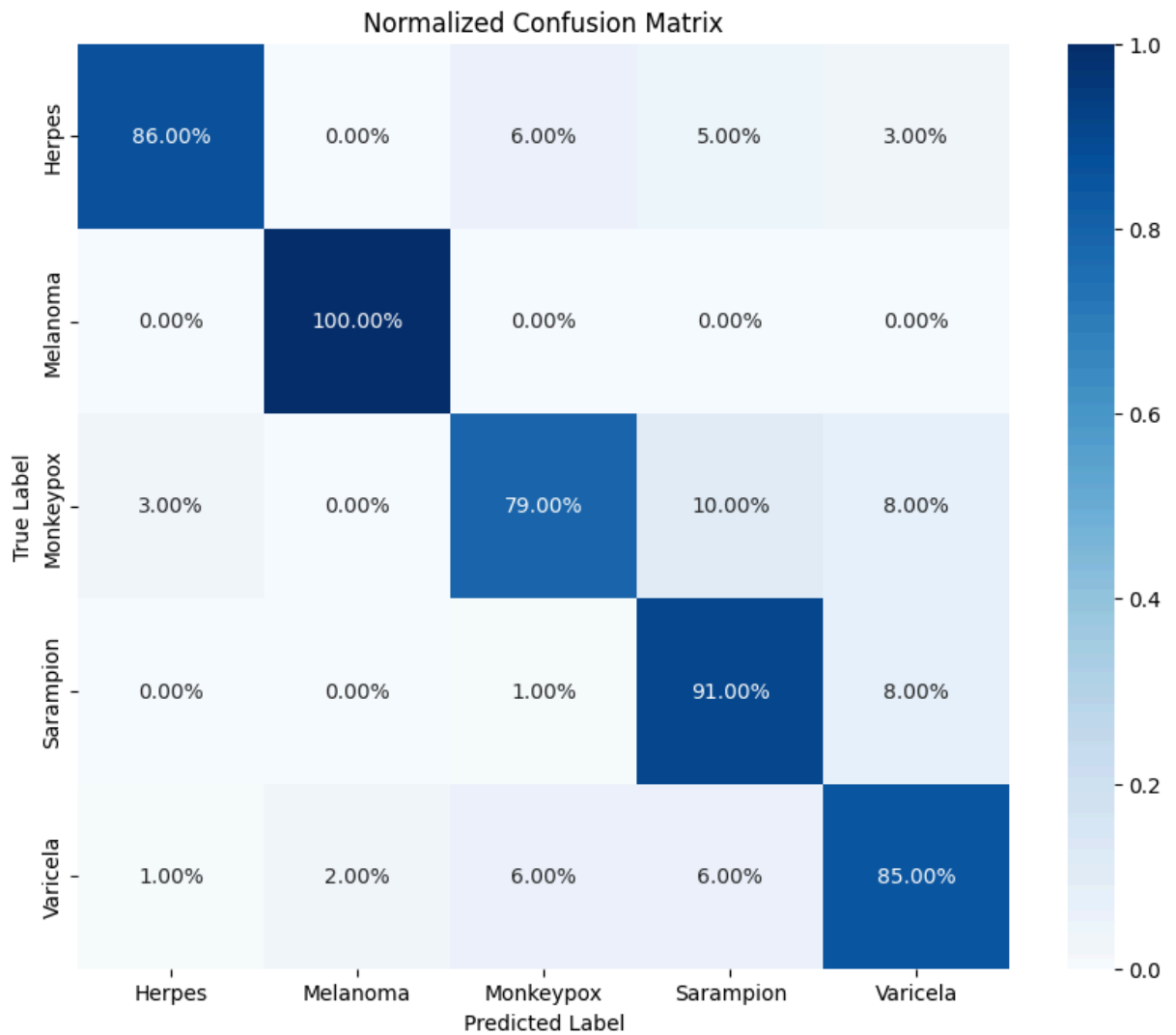
```

Accuracy: 0.8820

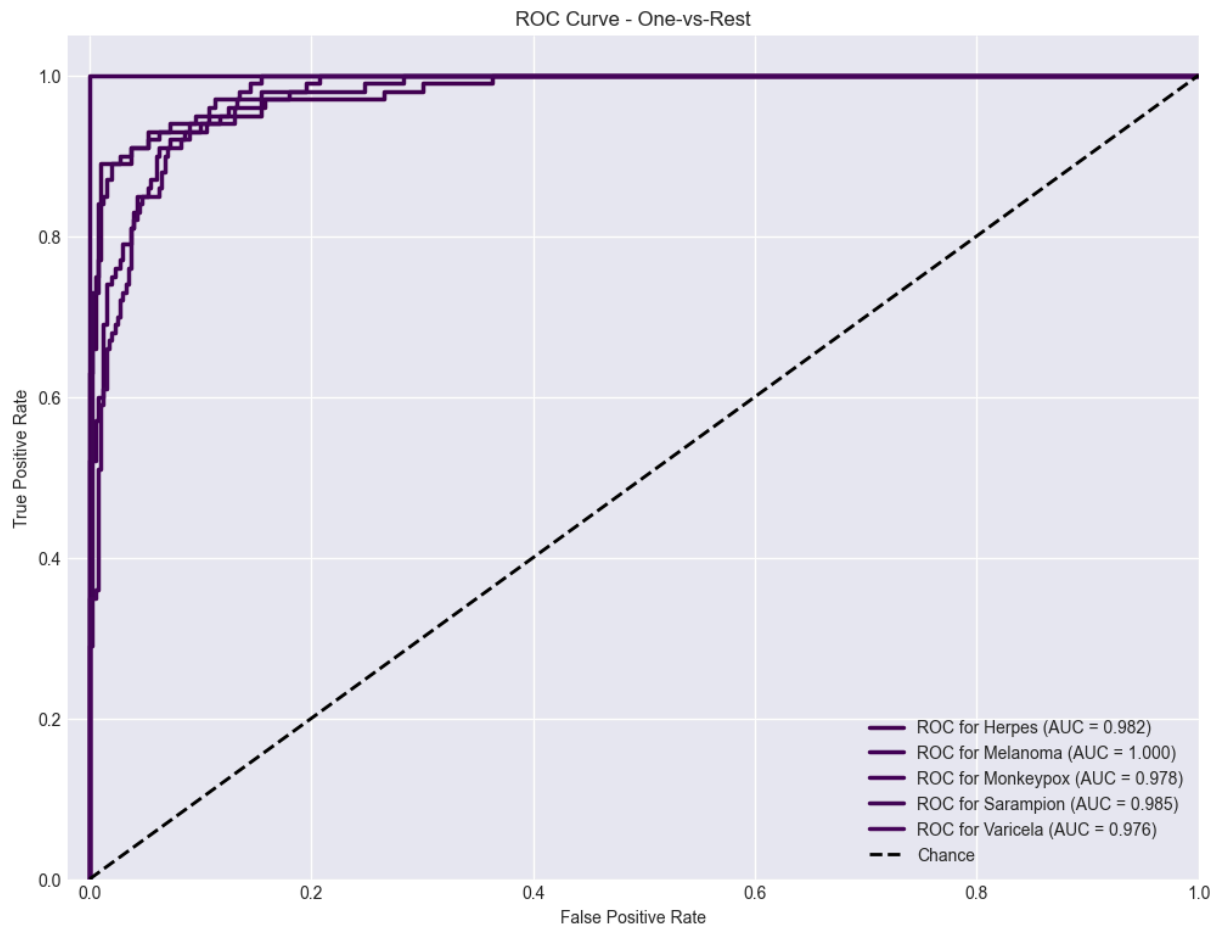
Macro ROC AUC: 0.9841

Cohen's Kappa: 0.8525

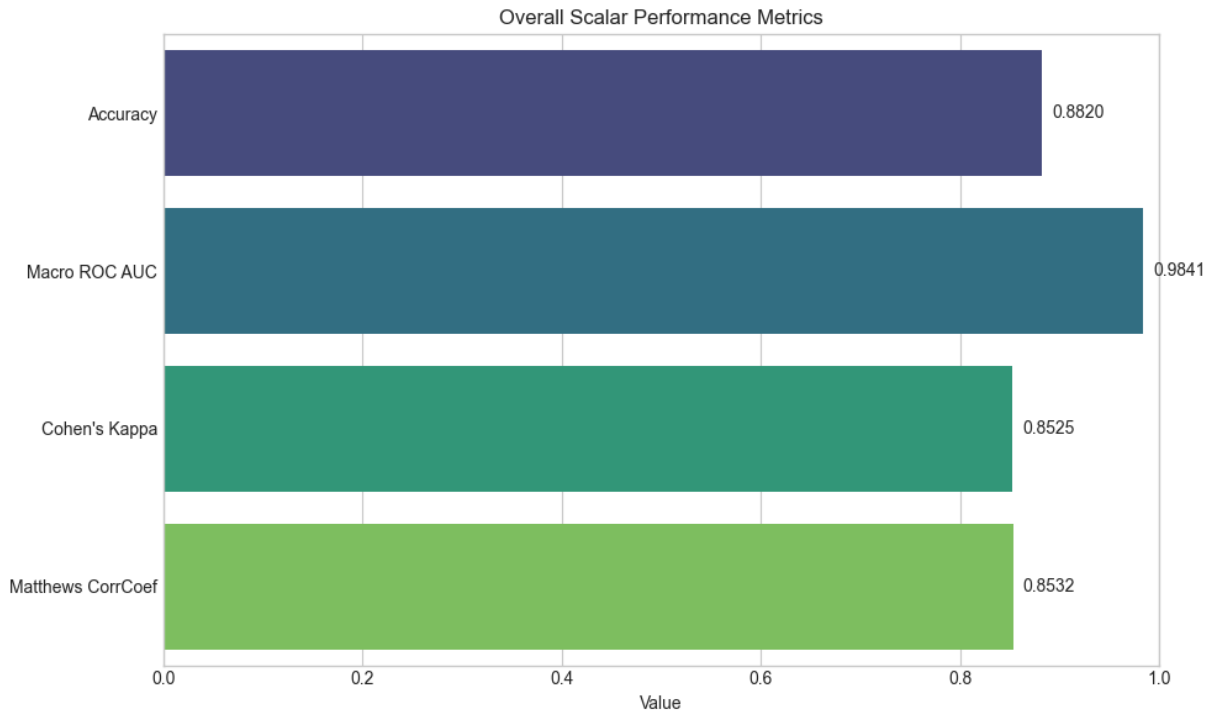
Matthews CorrCoef: 0.8532



Saved normalized confusion matrix to: full_results_ResNet50_ImageOnly_dropout\confusion_matrix.png



Saved ROC curves plot to: full_results_ResNet50_ImageOnly_dropout\roc_curves.png



Saved scalar metrics plot to: full_results_ResNet50_ImageOnly_dropout\scalar_metrics.png

Done.

In []: